

# Simulation Study for Pseudo-value Regression

FengQi (Cynthia) Song

2026-04-02

## Contents

|                                                                                                                                             |          |
|---------------------------------------------------------------------------------------------------------------------------------------------|----------|
| <b>1 Objective - evaluate the behavior of pseudo-value regression for recurrent event outcomes under the null and alternative settings.</b> | <b>1</b> |
| 1.1 Sanity check . . . . .                                                                                                                  | 1        |
| 1.2 Monte Carlo validation . . . . .                                                                                                        | 2        |
| 1.3 Sensitivity analysis(different evaluation time) . . . . .                                                                               | 3        |
| 1.4 Pseudo-value Regression Model . . . . .                                                                                                 | 5        |
| 1.5 Type I error . . . . .                                                                                                                  | 6        |
| 1.6 Power . . . . .                                                                                                                         | 8        |

---

## 1 Objective - evaluate the behavior of pseudo-value regression for recurrent event outcomes under the null and alternative settings.

### 1.1 Sanity check

```
library(MCC)

scenarios <- data.frame(
  scenario = c("baseline", "high_censor", "high_death", "high_both"),
  lambda_c = c(0.25, 0.40, 0.25, 0.40),
  lambda_d = c(0.25, 0.25, 0.40, 0.40)
)

lambda_c <- scenarios$lambda_c[1]
lambda_d <- scenarios$lambda_d[1]

source("01_generate_data.R")
t_eval <- 5
pseudo <- MCC::GenPseudo(data = dat, tau = t_eval)

str(pseudo)
```

```
## 'data.frame': 400 obs. of 3 variables:
## $ idx : num 1 2 3 4 5 6 7 8 9 10 ...
## $ psi : num -0.157 4.868 -0.504 0.547 -1.331 ...
## $ pseudo: num 1.17789 6.20286 0.83061 1.88146 0.00334 ...
```

```
head(pseudo)
```

```
##   idx      psi      pseudo
## 1    1 -0.1565125 1.177889409
## 2    2  4.8684576 6.202859550
## 3    3 -0.5037962 0.830605708
## 4    4  0.5470610 1.881462906
## 5    5 -1.3310659 0.003336005
## 6    6 -0.3334910 1.000910893
```

```
AX <- unique(dat[, c("idx", "A", "X")])
merged <- merge(AX, pseudo[, c("idx", "pseudo")], by = "idx")
```

```
tapply(merged$pseudo, merged$A, mean)
```

```
##           0           1
## 1.369004 1.299800
```

## 1.2 Monte Carlo validation

```
library(MCC)
lambda_c <- 0.25
lambda_d <- 0.25

source("01_generate_data.R")

set.seed(123)

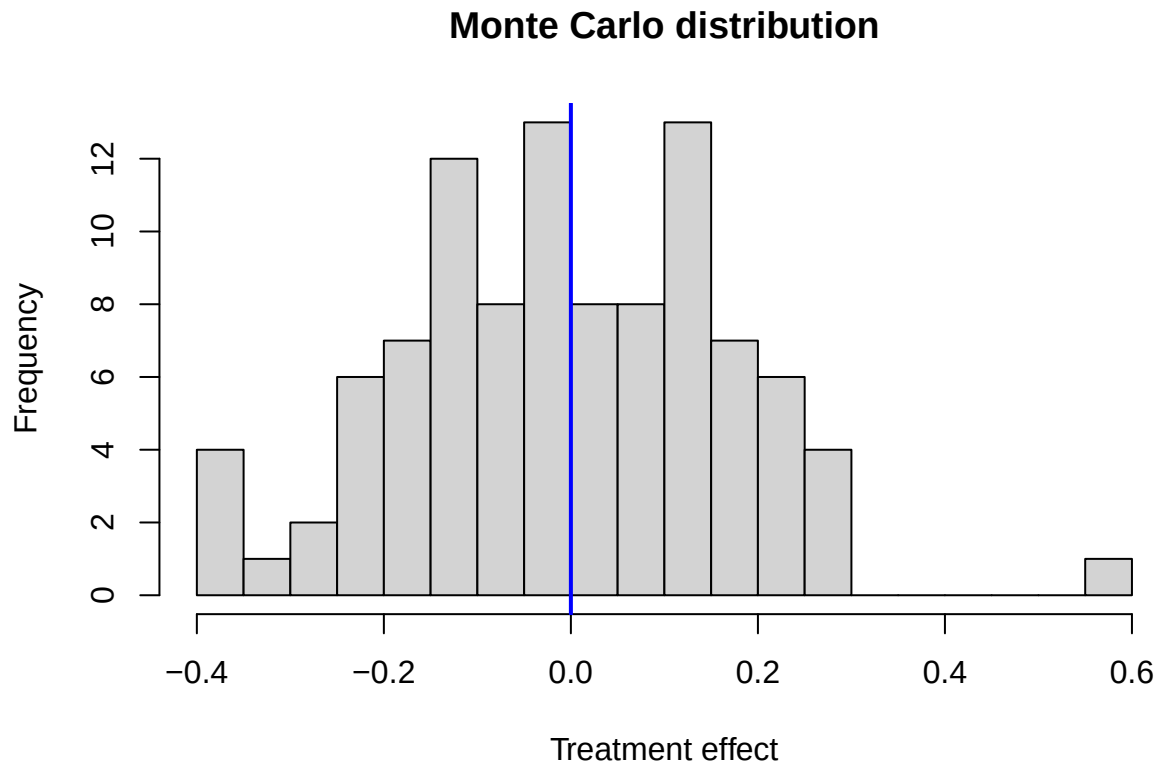
B <- 100
delta_hat <- numeric(B)

for (b in 1:B) {
  source("01_generate_data.R")
  pseudo <- MCC::GenPseudo(data = dat, tau = 5)

  AX <- unique(dat[, c("idx", "A")])
  merged <- merge(AX, pseudo[, c("idx", "pseudo")], by = "idx")

  delta_hat[b] <- mean(merged$pseudo[merged$A == 1]) -
    mean(merged$pseudo[merged$A == 0])
}

hist(delta_hat, breaks = 30,
      main = "Monte Carlo distribution",
      xlab = "Treatment effect")
abline(v = 0, col = "blue", lwd = 2)
```



This distribution is centered at zero, suggesting no bias

### 1.3 Sensitivity analysis(different evaluation time)

```
set.seed(123)

lambda_c <- 0.25
lambda_d <- 0.25

B <- 100
t_list <- c(2.5, 5, 7.5)

out_time <- data.frame(
  t_eval = t_list,
  mean_diff = NA_real_,
  sd_diff = NA_real_,
  se_diff = NA_real_,
  lower = NA_real_,
  upper = NA_real_
)

for (j in seq_along(t_list)) {
  t_eval <- t_list[j]
```

```

delta_tau <- numeric(B)

for (b in 1:B) {

  source("01_generate_data.R")

  pseudo_j <- MCC::GenPseudo(data = dat, tau = t_eval)
  AX <- unique(dat[, c("idx", "A")])
  merged_j <- merge(AX, pseudo_j[, c("idx", "pseudo")], by = "idx")

  delta_tau[b] <- mean(merged_j$pseudo[merged_j$A == 1]) -
    mean(merged_j$pseudo[merged_j$A == 0])
}

out_time$mean_diff[j] <- mean(delta_tau)
out_time$sd_diff[j] <- sd(delta_tau)
out_time$se_diff[j] <- out_time$sd_diff[j] / sqrt(B)
out_time$lower[j] <- out_time$mean_diff[j] - 1.96 * out_time$se_diff[j]
out_time$upper[j] <- out_time$mean_diff[j] + 1.96 * out_time$se_diff[j]
}

out_time

```

```

##   t_eval  mean_diff  sd_diff  se_diff    lower    upper
## 1    2.5 0.002129951 0.1232470 0.01232470 -0.02202647 0.02628637
## 2    5.0 0.006239875 0.1640164 0.01640164 -0.02590734 0.03838709
## 3    7.5 0.015527440 0.1874126 0.01874126 -0.02120544 0.05226032

```

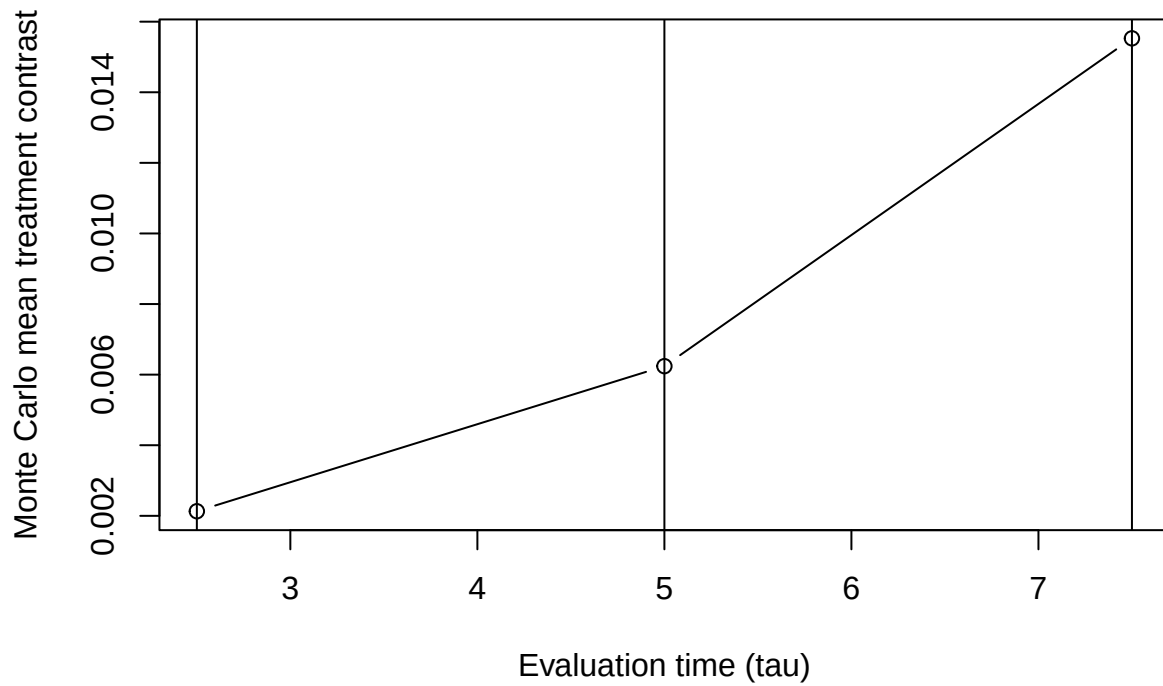
```

plot(out_time$t_eval, out_time$mean_diff, type = "b",
     xlab = "Evaluation time (tau)",
     ylab = "Monte Carlo mean treatment contrast")

arrows(
  x0 = out_time$t_eval,
  y0 = out_time$lower,
  x1 = out_time$t_eval,
  y1 = out_time$upper,
  angle = 90,
  code = 3,
  length = 0.05
)

abline(h = 0, col = "blue")

```



## 1.4 Pseudo-value Regression Model

```
# Single-run regression across tau

t_list <- c(2.5, 5, 7.5)
AX <- unique(dat[, c("idx", "A")])

reg_out <- data.frame(
  t_eval = t_list,
  beta_A = NA,
  p_A = NA
)

for (j in seq_along(t_list)) {
  t_eval <- t_list[j]
  pseudo_j <- MCC::GenPseudo(data = dat, tau = t_eval)
  reg_dat <- merge(AX, pseudo_j[, c("idx", "pseudo")], by = "idx")

  fit <- lm(pseudo ~ A, data = reg_dat)
  s <- summary(fit)$coefficients

  reg_out$beta_A[j] <- s["A", "Estimate"]
  reg_out$p_A[j] <- s["A", "Pr(>|t|)"]
}
```

```
reg_out
```

```
##   t_eval      beta_A      p_A
## 1    2.5 -0.08028671 0.4464848
## 2    5.0 -0.18090018 0.2763444
## 3    7.5 -0.19523399 0.3632168
```

## 1.5 Type I error

```
set.seed(123)

B <- 100
alpha <- 0.05
t_eval <- 2
alphaA <- 0

type1_result <- data.frame(
  scenario = scenarios$scenario,
  lambda_c = scenarios$lambda_c,
  lambda_d = scenarios$lambda_d,
  reject_rate = NA_real_,
  mean_beta = NA_real_,
  se_reject = NA_real_,
  lower = NA_real_,
  upper = NA_real_
)

for (j in seq_len(nrow(scenarios))) {

  lambda_c <- scenarios$lambda_c[j]
  lambda_d <- scenarios$lambda_d[j]

  reject <- logical(B)
  beta <- numeric(B)

  for (b in 1:B) {

    source("01_generate_data.R")

    pseudo_b <- MCC::GenPseudo(data = dat, tau = t_eval)

    AX <- unique(dat[, c("idx", "A")])
    reg_dat <- merge(AX, pseudo_b[, c("idx", "pseudo")], by = "idx")

    fit <- lm(pseudo ~ A, data = reg_dat)
    s <- summary(fit)$coefficients

    beta[b] <- s["A", "Estimate"]
    reject[b] <- s["A", "Pr(>|t|)"] < alpha
  }
}
```

```

type1_result$reject_rate[j] <- mean(reject)
type1_result$mean_beta[j] <- mean(beta)

type1_result$se_reject[j] <- sqrt(type1_result$reject_rate[j] *
                                  (1 - type1_result$reject_rate[j]) / B)

type1_result$lower[j] <- max(0,
                             type1_result$reject_rate[j] -
                             1.96 * type1_result$se_reject[j])

type1_result$upper[j] <- min(1,
                             type1_result$reject_rate[j] +
                             1.96 * type1_result$se_reject[j])
}

type1_result

##      scenario lambda_c lambda_d reject_rate  mean_beta se_reject      lower
## 1 baseline      0.25      0.25         0.08  0.007738530 0.02712932 0.026826533
## 2 high_censor    0.40      0.25         0.04 -0.003883853 0.01959592 0.001592001
## 3 high_death     0.25      0.40         0.03  0.002945743 0.01705872 0.000000000
## 4 high_both      0.40      0.40         0.10  0.009785891 0.03000000 0.041200000
##      upper
## 1 0.1331735
## 2 0.0784080
## 3 0.0634351
## 4 0.1588000

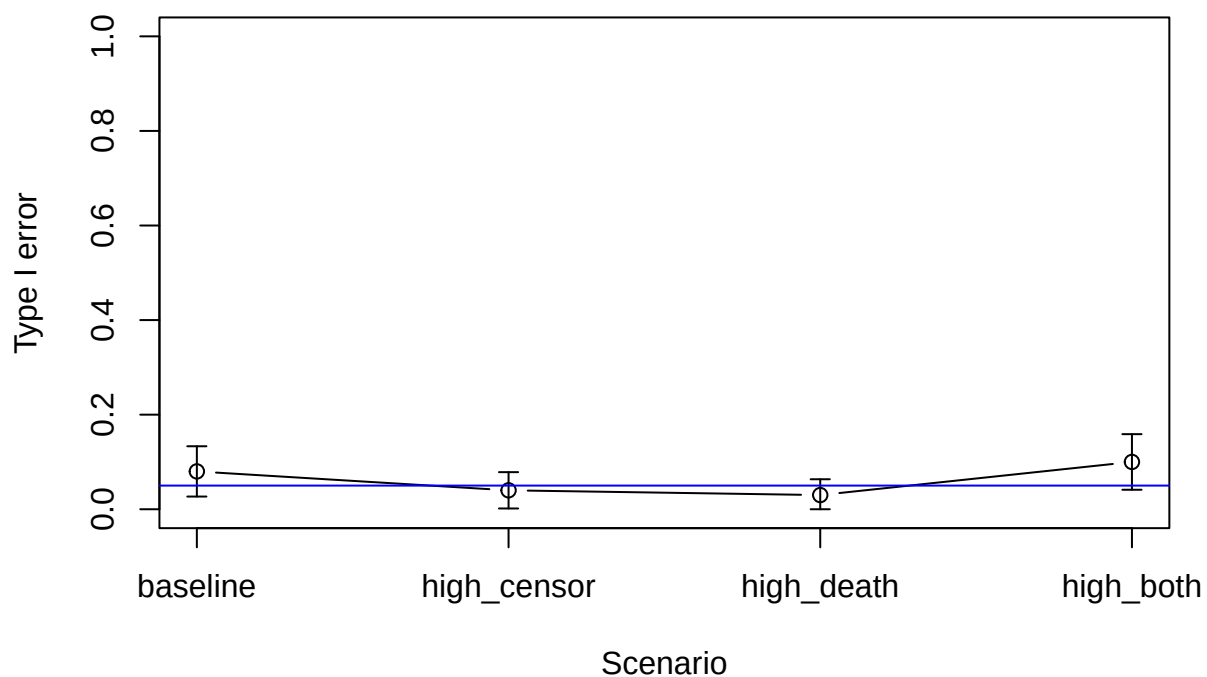
plot(seq_len(nrow(type1_result)), type1_result$reject_rate, type = "b",
     xaxt = "n",
     ylim = c(0, 1),
     xlab = "Scenario",
     ylab = "Type I error")

axis(1, at = seq_len(nrow(type1_result)), labels = type1_result$scenario)

arrows(
  x0 = seq_len(nrow(type1_result)),
  y0 = type1_result$lower,
  x1 = seq_len(nrow(type1_result)),
  y1 = type1_result$upper,
  angle = 90,
  code = 3,
  length = 0.05
)

abline(h = 0.05, col = "blue")

```



## 1.6 Power

```
## Power, Type II error

set.seed(123)

B <- 100
alpha <- 0.05
t_eval <- 2
alphaA <- log(1.2)

power_out <- data.frame(
  scenario = scenarios$scenario,
  lambda_c = scenarios$lambda_c,
  lambda_d = scenarios$lambda_d,
  power = NA_real_,
  mean_beta = NA_real_,
  se_power = NA_real_,
  lower = NA_real_,
  upper = NA_real_
)

# added
risk_summary <- data.frame(
```



```

scenario = scenarios$scenario,
risk_prop = NA_real_
)

for (j in seq_len(nrow(scenarios))) {

  lambda_c <- scenarios$lambda_c[j]
  lambda_d <- scenarios$lambda_d[j]

  reject <- logical(B)
  beta <- numeric(B)

  for (b in 1:B) {

    source("01_generate_data.R")

    # new
    risk_prop <- mean(tapply(dat$time > t_eval, dat$idx, any))

    if (b == 1) {
      risk_summary$risk_prop[j] <- risk_prop
    }

    pseudo_b <- MCC::GenPseudo(data = dat, tau = t_eval)
    AX <- unique(dat[, c("idx", "A")])
    reg_dat <- merge(AX, pseudo_b[, c("idx", "pseudo")], by = "idx")

    fit <- lm(pseudo ~ A, data = reg_dat)
    s <- summary(fit)$coefficients

    beta[b] <- s["A", "Estimate"]
    reject[b] <- s["A", "Pr(>|t|)" ] < alpha
  }

  power_out$power[j] <- mean(reject)
  power_out$mean_beta[j] <- mean(beta)

  power_out$se_power[j] <- sqrt(power_out$power[j] * (1 - power_out$power[j]) / B)
  power_out$lower[j] <- max(0, power_out$power[j] - 1.96 * power_out$se_power[j])
  power_out$upper[j] <- min(1, power_out$power[j] + 1.96 * power_out$se_power[j])
}

power_out

```

```

##      scenario lambda_c lambda_d power mean_beta se_power lower upper
## 1 baseline      0.25      0.25  0.28  0.1326356 0.04489989 0.19199622 0.3680038
## 2 high_censor    0.40      0.25  0.17  0.1175108 0.03756328 0.09637597 0.2436240
## 3 high_death     0.25      0.40  0.25  0.1141982 0.04330127 0.16512951 0.3348705
## 4 high_both      0.40      0.40  0.24  0.1214783 0.04270831 0.15629171 0.3237083

```

```

risk_summary

```

```

##      scenario risk_prop

```

```
## 1    baseline    0.3375
## 2 high_censor    0.2650
## 3 high_death    0.2550
## 4   high_both    0.2175
```

```
plot(seq_len(nrow(power_out)), power_out$power, type = "b",
     xaxt = "n",
     ylim = c(0, 1),
     xlab = "Scenario",
     ylab = "Power")

axis(1, at = seq_len(nrow(power_out)), labels = power_out$scenario)

arrows(
  x0 = seq_len(nrow(power_out)),
  y0 = power_out$lower,
  x1 = seq_len(nrow(power_out)),
  y1 = power_out$upper,
  angle = 90,
  code = 3,
  length = 0.05
)
```

